

深度学习 C 实现笔记

HCX

2026 年 08 月 05 日

1 推导

1.1 反向传播

根据神经网络，可以记 W_i , b_i 为第 i 层到第 $i+1$ 层的神经元权重，记中间结果激活值为 a_i ，则对每一层都有

$$a_{i+1} = \sigma(W_i a_i + b_i) \triangleq \sigma(t_i)$$

其中， $\sigma(\cdot)$ 为激活函数， t_i 为 weighted-sum 值。根据损失函数 $\ell(\hat{y}, y) = \frac{1}{n}(\hat{y} - y)^T(\hat{y} - y)$ ，则由 $\hat{y} = a_{n+1}$ 得

$$\frac{\partial \ell}{\partial a_{n+1}} = \frac{\partial \ell}{\partial \hat{y}} = \frac{2}{n}(\hat{y} - y) = \frac{2}{n}(a_{n+1} - y)$$

$$\frac{\partial a_{i+1}}{\partial a_i} = \frac{\partial a_{i+1}}{\partial t_i} \frac{\partial t_i}{\partial a_i} = \sigma'(t_i) \odot W_i$$

$$\frac{\partial t_i}{\partial W_i} = a_i^T, \quad \frac{\partial t_i}{\partial b_i} = I$$

则定义误差项 $\delta_i = \frac{\partial \ell}{\partial t_i}$ 。在输出层 $i = n$ 处，由 $\hat{y} = a_{n+1} = \sigma(t_n)$ 得

$$\delta_n = \frac{\partial \ell}{\partial a_{n+1}} \odot \sigma'(t_n) = \frac{2}{n}(a_{n+1} - y) \odot \sigma'(t_n)$$

其中 $\odot$ 表示逐元素（Hadamard）乘积。递推得

$$\delta_i = \sigma'(t_i) \odot (W_{i+1}^T \delta_{i+1}), \quad i = n-1, \dots, 1$$

结合 $\frac{\partial t_i}{\partial W_i} = a_i^T$ 与 $\frac{\partial t_i}{\partial b_i} = I$ ，即得各层参数的梯度

$$\frac{\partial \ell}{\partial W_i} = \delta_i a_i^T, \quad \frac{\partial \ell}{\partial b_i} = \delta_i$$

2 编程经验

2.1 C 语言相关

2.1.1 X-macro

```
// ===== Activation System =====

static inline nn_real sigmoidf(nn_real x) {
    return 1.0f / (1.0f + expf(-x));
}
```

```

static inline nn_real reluf(nn_real x) {
    return x > 0.0f ? x : 0.0f;
}

static inline nn_real geluf(nn_real x) {
    return 0.5f * x * (1.0f + tanhf(sqrtf(2.0f / M_PI) * (x + 0.044715f *
powf(x, 3.0f))));
}

#define ACTIVATIONS \
    X(ACbold(t)_SIGMOID, sigmoidf) \
    X(ACbold(t)_RELU,      reluf)      \
    X(ACbold(t)_GELU,      geluf)

#define X(id, name) id,
typedef enum { ACTIVATIONS } ActivationType;
#undef X

static inline nn_real activate(ActivationType type, nn_real x) {
    switch (type) {
        #define X(id, name) case id: return name(x);
        ACTIVATIONS
        #undef X
    }
    return x;
}

```

X-macro 技术可以用来定义一组相关的宏或函数，避免重复代码，提高可维护性。

2.1.2 视图式的矩阵内存管理

```

typedef struct {
    size_t rows;
    size_t cols;
    size_t stride;
    nn_real *elements;
} Matrix;

```

使用 `stride` 来表示每行的跨度，这样可以方便地创建子矩阵或行视图，而不需要复制数据。